

Ultra Ethernet Transport (SES/PDS) in ns-3: Complete Project Documentation

Sayan Das

July 2026

Contents

1. Project overview	3
1.1 Why Ultra Ethernet exists	3
1.2 Delivery modes	3
2. Repository layout	3
3. Building	4
3.1 Prerequisites	4
3.2 Commands	4
4. Running everything	5
4.1 Verification suite (run this first)	5
4.2 uet-complete-demo: protocol walkthrough	5
4.3 uet-hpc-ai-profiles: workload profiles	5
4.4 uet-network-sim: the quantitative benchmark	6
4.5 uet-ses-pds-demo: deterministic drop injection	6
4.6 uet-advanced-demo and uet-phase4-rdma-integration	6
4.7 Reproducing the results tables	6
5. Implementation deep dive	6
5.1 Layered architecture	6
5.2 SES wire formats (<code>uet-ses-header.{h,cc}</code>)	7
5.3 PDS wire formats (<code>uet-pds-header.{h,cc}</code>)	8
5.4 PDC manager (<code>uet-pdc.{h,cc}</code>)	8
PDC context and state machine	8
Establishment sequence	9
PSN space (<code>PdcPsnSpace</code>)	9
Receive-side check order (<code>ProcessRxRequest</code>)	9
5.5 Transaction engine (<code>uet-ses-pds-engine.{h,cc}</code>)	10
TX path	10
RX dispatch	10
Loss recovery	11
Completion semantics	11
5.6 NACK handling	11
5.7 RDMA hardware bridge (<code>rdma-hw-uet-integration.{h,cc}</code>)	11

5.8 Design decisions worth knowing	12
6. Configuration reference (engine defaults)	12
7. Results	12
8. Troubleshooting	13
9. Limitations and future work	13
10. Glossary	13

1. Project overview

This project implements the transport layer defined by the Ultra Ethernet Consortium (UEC) in **UE-Specification 1.0** inside the **ns-3.36.1** discrete-event network simulator (ns-3-alibabacloud / SimAI tree). Two spec layers are covered:

- **SES, Semantic Sublayer (spec section 3.4)**: message semantics, opcodes, addressing, and seven wire header formats.
- **PDS, Packet Delivery Sublayer (spec section 3.5)**: reliability, ordering, connection management (Packet Delivery Contexts), sequence-number spaces, ACK/NACK machinery, and twelve wire packet formats.

The stack runs end-to-end over real ns-3 UDP sockets and point-to-point links. Under injected packet loss it recovers via selective acknowledgment (SACK), retransmission timeouts (RTO), and NACK handling; 100 percent of messages complete at every tested loss rate from 0 to 5 percent. Correctness is enforced by a deterministic test suite (11 scenarios, 86 assertions).

1.1 Why Ultra Ethernet exists

RoCEv2, today’s dominant RDMA transport, assumes a lossless fabric (built with Priority Flow Control) and recovers from any loss with go-back-N retransmission. At AI/HPC scale this causes head-of-line blocking, congestion trees, PFC deadlock risk, and massive bandwidth waste on rare loss. UET removes the lossless requirement:

- packets may arrive out of order (enables multipath packet spraying),
- receivers acknowledge selectively (64-bit SACK bitmap),
- connection state is ephemeral (PDCs, established in-band at zero RTT cost),
- applications choose the ordering/reliability they need per message.

1.2 Delivery modes

Mode	Reliability	Ordering	Connection	Typical use
RUD	Reliable	Unordered	PDC	RDMA write/read, allreduce
ROD	Reliable	Ordered	PDC	MPI collectives, atomics
RUDI	Reliable	Unordered, idempotent	none (pkt_id)	small datagrams
UUD	Unreliable	Unordered	none	best-effort telemetry

2. Repository layout

```
.
|-- README.md                project front page (results, architecture)
|-- docs/
|   |-- PROJECT_DOCUMENTATION.md this document
|   |-- GUIDE.md              shorter build/run guide
|   |-- REPORT.md             audit findings and evaluation methodology
|-- scripts/
|   |-- build.sh              configure + build everything
```

```

|   |-- run_tests.sh           86-check verification suite
|   |-- run_demos.sh          all demo programs
|   `-- run_experiments.sh    reproduce the results tables
|-- dashboard/                static web visualization (open index.html)
|-- legacy/                   superseded documents kept for history
`-- simulation/               ns-3.36.1 tree
    |-- scratch/              UET programs (see section 4)
    `-- src/point-to-point/model/
        |-- uet-ses-header.{h,cc}    SES wire formats
        |-- uet-pds-header.{h,cc}    PDS wire formats
        |-- uet-pdc.{h,cc}          PDC manager, PSN spaces
        |-- uet-ses-pds-engine.{h,cc} transaction engine, RT0, reorder buffer
        |-- uet-pds-control-packet.{h,cc}
        |-- uet-pds-target-state.{h,cc}
        `-- rdma-hw-uet-integration.{h,cc}

```

3. Building

3.1 Prerequisites

Requirement	Version	Notes
C++ compiler	clang 12+ or gcc 9+ (C++17)	Apple clang works
CMake	3.10 or newer	
Python 3	any recent	used by the ns3 wrapper
OS	macOS or Linux	developed on macOS (Apple Silicon)

No external libraries are needed; ns-3 is vendored in `simulation/`.

3.2 Commands

```

scripts/build.sh           # incremental build (recommended)
scripts/build.sh --clean   # wipe artifacts, rebuild from scratch

```

Manual equivalent:

```

cd simulation
./ns3 configure --build-profile=debug
./ns3 build           # everything
./ns3 build scratch/uet-tests # one target only

```

A cold build compiles the whole ns-3 tree (several minutes). Incremental builds touching only UET files take seconds. Binaries land in `simulation/build/scratch/` named `ns3.36.1-<program>-debug`.

Note on build profiles: the binary suffix follows the profile (`-debug`, `-optimized`, `-default`). All commands in this document assume the debug profile.

4. Running everything

4.1 Verification suite (run this first)

```
scripts/run_tests.sh
# or directly:
./simulation/build/scratch/ns3.36.1-uet-tests-debug
```

Expected final line:

UET test suite: 86/86 checks passed -- ALL TESTS PASSED

Exit code 0 means success. The suite is deterministic (fixed seed).

Test	Proves
T01	All 7 SES formats survive serialize/deserialize round-trips
T02	All PDS formats round-trip (REQ, ACK, ACK_CC+SACK, NACK, CP, RUDI, UUD)
T03	PDC SYN establishment, TPDCID learning, PDC reuse across messages
T04	Multi-packet message delivers fully; completion fires exactly once
T05	Dropped request healed by RTO retransmission
T06	Dropped ACK: duplicate is re-ACKed, payload not delivered twice
T07	ROD delivers in order under a mid-message drop
T08	RUDI retransmits by pkt_id; target deduplicates
T09	UUD completes immediately, leaves no reliability state
T10	SACK bitmap construction; CACK-anchored window acceptance
T11	Retry-budget exhaustion reports failure exactly once

4.2 uet-complete-demo: protocol walkthrough

```
scripts/run_demos.sh complete
```

Four engines (N0..N3) on an in-process fabric. Prints every SES and PDS header format with sizes and spec table references, walks a full PDC establishment for a 3 x 4 KB RUD write, runs ROD/RUDI/UUD examples, demonstrates PSN space mechanics including a worked SACK bitmap and the spec Table 3-43 ACK_PSN offset vectors, and ends with per-node statistics. Every sender should log a completion: `ack msgId=... OK rc=0x0`.

4.3 uet-hpc-ai-profiles: workload profiles

```
scripts/run_demos.sh hpc
```

Eight nodes exercise the three UET profiles from spec section 2.2.2: AI Base (ring allreduce, RUD writes), AI Full (tagged rendezvous sends, deferrable sends, ROD atomics), HPC (MPI-style scatter/gather over ROD, SHMEM put/get over RUD), then print the profile feature matrix.

4.4 uet-network-sim: the quantitative benchmark

Runs the engine over real ns-3 UDP sockets on a point-to-point link, with optional loss injection.

```
cd simulation
./build/scratch/ns3.36.1-uet-network-sim-debug \
  --numMsgs=2000 --msgSize=8192 --mode=RUD --lossRate=0.01 --seed=1
```

Flag	Default	Meaning
--msgSize	65535	application message size (bytes)
--numMsgs	10000	number of messages
--dataRate	200Gbps	link rate
--delay	100ns	propagation delay
--mode	RUD	RUD, ROD, RUDI, UUD
--seed	1	RNG seed; same seed = identical run
--lossRate	0	per-packet receive drop probability, both directions
--simTime	10.0	simulation stop (seconds)

Output: goodput vs target, reliability accounting (sent / delivered / completed / failed), delivery-latency percentiles, engine counters (retransmissions, NACKs, PDCs), and an assessment line that prints FAIL if completion is below 100 percent. Detailed PDC state is written to `pdc_state_dump.log`.

4.5 uet-ses-pds-demo: deterministic drop injection

```
cd simulation
./build/scratch/ns3.36.1-uet-ses-pds-demo-debug \
  --messageBytes=8000 --mtuBytes=1000 \
  --dropReqPsns=1,3 --dropAckPsns=0 --mode=rud
```

Drops exactly the listed request/ACK ordinals on first transmission and lets recovery heal them. Expected: `retx=2`, `deliveredBytes=8000`, completion `rc=0x0`. Also supports `--dropRequestRate`, `--dropAckRate` (seeded probabilistic drops) and `--maxRetries`.

4.6 uet-advanced-demo and uet-phase4-rdma-integration

```
scripts/run_demos.sh advanced  # control packets, target state scenarios
scripts/run_demos.sh phase4    # RDMA bridge TX/RX walkthrough
```

4.7 Reproducing the results tables

```
scripts/run_experiments.sh          # seed 1 (the published tables)
SEED=2 scripts/run_experiments.sh  # robustness check
```

Writes one file per configuration to `results/` (RUD and ROD at loss 0 / 0.001 / 0.01 / 0.05, plus RUDI at 0.01) and prints a completion/goodput summary.

5. Implementation deep dive

5.1 Layered architecture

+-----+

```

| Applications / demos / benchmark / test harness |
+-----+
| UetSesPdsEngine (one per endpoint) |
|   TX: Send() -> fragment -> SES hdr -> PDS hdr -> wire cb |
|   RX: ProcessRxPacket() -> dispatch by PDS type |
|   RTO timer | ROD reorder buffer | RUDI dedup | completions |
+-----+
| UetPdcManager |
|   PDC contexts (IPDCID/TPDCID, state machine) |
|   PdcPsnSpace (Start_PSN, CACK, CLEAR, window, SACK, |
|               outstanding map with stored wire packets) |
+-----+
| Wire formats: UetSesHeader (7), UetPdsHeader (12) |
|   ns3::Header subclasses, bit-exact to spec tables |
+-----+
| ns-3 substrate: UDP sockets or in-process fabric router, |
|   PointToPoint devices, queues, RateErrorModel loss |
+-----+

```

The engine is transport-pure: its only output is a callback `void(Ptr<Packet>, uint32_t dstFa)`. The benchmark maps `dstFa` to a UDP `SendTo`; demos route through an in-process table; the test harness interposes drop filters. The same engine binary code runs in all three environments.

5.2 SES wire formats (`uet-ses-header.{h,cc}`)

One class, `UetSesHeader`, serializes seven layouts selected by `SetFormat()`:

Format	Size	Spec table	Content highlights
REQUEST_STD SOM=1	44 B	3-8	opcode, flags, msg id, JobID(24b), PIDonFEP(12b), resource index(12b), buffer offset(64b), initiator, match bits, header data, request length
REQUEST_STD SOM=0	32 B	3-9	continuation: payload_length(14b), message_offset(32b) replace header_data
REQUEST_SMALL	20 B	3-10	optimized non-matching
REQUEST_MEDIUM	28 B	3-14	small message / small RMA with match bits
RESPONSE	12 B	3-11	list(2b), return code(6b), modified length
RESPONSE_DATA20	20 B	3-12	read-return data, response + original msg ids
RESPONSE_DATA256 SMALL	256 B	3-13	optimized, keyed by original request PSN

Flags carried by all request formats: `som`, `eom` (message framing), `dc` (delivery-complete), `ie` (initiator error), `rel` (relative addressing), `hd` (header-data present). An optional 4-byte atomic extension header follows request formats when `hasAtomic` is set.

Parsing subtlety worth knowing: the STD deserializer reads the `som` bit from the wire (bit 0 of byte 1, identical position in both layouts) to select the 44-byte vs 32-byte layout. It must never trust caller state: continuation headers parsed with the SOM=1 layout silently swallow 12 payload bytes.

5.3 PDS wire formats (`uet-pds-header.{h,cc}`)

One class, `UetPdsHeader`, covers all packet types, selected by `prologue.pdsType`:

Type	Size	Purpose
RUD_REQ / ROD_REQ	12 B	data request: <code>clear_psn_offset</code> (16b signed), PSN(32b), <code>spdcid</code> , <code>dpdcid</code>
RUD_CC_REQ / ROD_CC_REQ	16 B	request + 32b CC state
ACK	12 B	<code>ack_psn_offset</code> or <code>probe_opaque</code> , CACK_PSN, <code>spdcid</code> , <code>dpdcid</code>
ACK_CC	32 B	ACK + <code>cc_type/flags</code> , MPR, <code>sack_psn_offset</code> , 64b SACK bitmap, 64b CC state
ACK_CCX	44 B	ACK + 128b extended CC state
NACK	16 B	nack code, vendor code, nack PSN, PDCIDs, 32b payload
NACK_CCX	28 B	NACK + 96b extended CC state
CP	16 B	control packet: <code>ctl_type</code> in prologue, PSN, PDCIDs, 32b payload
RUDI_REQ / RUDI_RESP	6 B	prologue + 32b <code>pkt_id</code>
UUD_REQ	4 B	prologue + reserved

Prologue (2 bytes, every packet): `type`(5b) | `next_hdr_or_ctl_type`(4b) | `flags`(7b). Flag interpretation depends on packet class: requests carry `retx/ar/syn`; ACKs carry `m` (ECN-marked), `retx`, `p` (probe reply), `req` (requests a clear/close); NACKs carry `m/retx/nt`; CPs carry `isrod/retx/ar/syn`.

Computed values (all offsets signed 16-bit, wraparound-safe):

- `CLEAR_PSN` = `psn` + `clear_psn_offset`
- `ACK_PSN` = `cack_psn` + `ack_psn_offset`
- `SACK_PSN` = `cack_psn` + `sack_psn_offset` (bitmap base)

SYN overload (spec 3.5.8.2): while the initiator does not yet know the TPDCID, the 16-bit `dpdcid` field of requests carries `{pdc_info(4b), psn_offset(12b)}` instead, so the target can compute `Start_PSN = psn - psn_offset` from any SYN packet, not only the first. `SetSynDpdcid()/GetSynDpdcid()` implement the packing.

NACK codes: all 20 codes of Table 3-58 are defined with their error class (NORMAL / PDC_FATAL / PDC_ERR) and required source action; see section 5.6.

5.4 PDC manager (`uet-pdc.{h,cc}`)

PDC context and state machine

```
CLOSED -> CREATING -> ESTABLISHED -> QUIESCE -> ACK_WAIT -> CLOSE
      (SYN sent)   (TPDCID known) (draining)
```

A `PdcContext` holds both directions, identifiers, mode (RUD/ROD), the SYN flag, RTO configuration, and a transition log used by the demos.

Establishment sequence

Initiator	Target
-----	-----
alloc_pdc: IPDCID, random Start_PSN	
REQ psn=S, syn=1, spdcid=IPDCID, dpdcid={pdc_info,psn_offset} ----->	unknown (srcFA, spdcid)? -> allocate TPDCID, state ESTABLISHED -> record PSN
	<----- ACK_CC spdcid=TPDCID, dpdcid=IPDCID, cack, SACK
learn TPDCID, syn=0, state ESTABLISHED	
REQ psn=S+1, dpdcid=TPDCID ----->	direct lookup by (srcFA, spdcid)

Target-side lookup is keyed (source fabric address, spdcid); initiator-side by IPDCID (dpdcid echoed in ACKs/NACKs).

PDC reuse rule: FindExistingPdc matches contexts in ESTABLISHED *or* CREATING state. Messages queued while the handshake is in flight share the pending PDC; without this, pipelined workloads exhaust the PDC pool (256 default) immediately.

PSN space (PdcPsnSpace)

Per direction:

- TX: start_psn, next_tx_psn, cack_psn (all below are delivered), clear_psn, and outstanding: an ordered map PSN -> TxPkt { retry counts, RTO deadline, owning message id, stored wire packet copy }.
- RX: received_psn (dedup + SACK source), expected_rx_psn (ROD ordering), rx_cack_psn (highest in-order PSN received).

Key operations:

- AssignTxPsn() allocates the next PSN and creates the outstanding entry.
- OnAckReceived(ackPsn, cack, sackBitmap, sackBase, &released) releases the explicit PSN, everything cumulative below cack, and every SACKed PSN; released TxPkts are returned so the engine can drive per-message completion.
- OnPktReceived(psn, isRod) inserts into received_psn, advances expected_rx_psn (past any buffered consecutive PSNs) and rx_cack_psn.
- BuildSackBitmap(&base) sets base = rx_cack_psn + 1 and marks bit i for each received base + i (64 bits).
- InExpectedWindow(psn): accepted iff psn - (rx_cack_psn + 1) < MP_RANGE (unsigned arithmetic, so wraparound-safe). The window is **CACK-anchored**; anchoring at the highest-seen PSN would reject retransmissions of dropped packets.
- Start_PSN generation uses an internal xorshift PRNG seeded from the fabric address, guaranteeing the spec's "at least 2¹⁶ from last use" rule and full reproducibility.

Receive-side check order (ProcessRxRequest)

Order is semantics, not style:

1. trimmed? -> NACK_TRIMMED

2. SYN handling / context lookup (unknown non-SYN -> NACK_INV_DPDCID; source address mismatch -> NACK_PDC_HDR_MISMATCH)
3. **duplicate check** (`psn <= rx_cack` or already in `received_psn`) -> mark duplicate, process piggybacked CLEAR, return success so the caller **re-ACKs**. The duplicate usually means the original ACK was lost; NACKing it deadlocks recovery.
4. window check -> NACK_PSN_OOR_WINDOW
5. accept; record PSN; process CLEAR_PSN; return whether the payload is new.

ROD out-of-order arrivals inside the window are **accepted and SACKed**, not NACKed; ordering is enforced at delivery time by the engine's reorder buffer (section 5.5). NACK_ROD_OOO remains reserved for SYN-time violations.

5.5 Transaction engine (`uet-ses-pds-engine.{h,cc}`)

TX path

`Send(dstFa, tc, mode, opcode, totalBytes, bufOffset, jobId, srcPid, dstRi, gtd):`

1. RUD/ROD: find-or-create the PDC for the tuple; RUDI/UUD are PDC-less.
2. Create per-message state (`UetSendMessage`): SES header template, byte counters, `pendingPkts`, `notified` flag.
3. `Pump()`.

`Pump()` sends every chunk the window allows, across all pending messages:

- fragments at the configured MTU; sets `som/eom`, `message_offset`, `payload_length`;
- RUD/ROD chunks get a PDS header via `BuildTxReqHeader` (PSN assignment, SYN handling, `clear_psn_offset`) and the **full wire packet copy is stored** in the outstanding entry together with the owning message id and an RTO deadline;
- RUDI chunks carry a fresh `pkt_id` and are tracked in `m_rudiOutstanding`;
- UUD completes immediately at send time (no state retained).

Pump is **reentrancy-safe**: with a synchronous in-process fabric, a wire send can deliver an ACK inline, which completes and erases messages and calls Pump again. The loop therefore iterates over an ID snapshot, re-finds message state after every wire send, and defers nested invocations via a flag.

RX dispatch

`ProcessRxPacket(pkt, srcFa)` removes the PDS header and branches:

- **Request** -> `HandleRxRequest`: PDC-manager checks (section 5.4), then SES parsing. New ROD payloads enter the **reorder buffer** (PDC -> psn -> chunk) and are released to the application strictly in PSN order as `expected_rx_psn` advances; RUD delivers immediately. An ACK_CC (CACK + SACK + TPDCID) is returned for new packets *and* duplicates.
- **ACK** -> learn TPDCID if in SYN, release outstanding PSNs (cumulative + SACK), decrement owning messages' `pendingPkts`, fire completions, then `Pump()` because window space opened.
- **NACK** -> section 5.6.
- **RUDI_REQ** -> dedup on (`srcFa, pkt_id`), deliver if new, always answer RUDI_RESP.
- **RUDI_RESP** -> release the tracked packet, complete the message when all packets confirmed.
- **CP** -> control packet processing (CLEAR advances the clear window; CLOSE transitions/frees the PDC).
- **UUD** -> deliver payload, no state.

Loss recovery

- **RTO**: a periodic engine event (period = initial RTO, 100 us default in the benchmark) scans outstanding packets whose deadline passed. Each retransmission resends the stored wire copy and pushes the deadline out by $rto * 2^{\text{retry_cnt}}$. After `maxRetries` (default 7) the owning message **fails explicitly**: completion callback with `ok=false`, counters, and full purge of the message's remaining state so nothing retransmits as an orphan.
- **SACK**: ACK_CC bitmaps release out-of-order-received packets so only true losses wait for the RTO.
- **NACK fast path**: see table below. A safety valve caps NACK-driven retransmits of one PSN (32) so a zero-latency demo fabric cannot ping-pong; the RTO path remains the backstop.

Completion semantics

Exactly-once, via a single choke point (`CompleteMessage`): guarded by the `notified` flag, fires the user callback, updates counters, unmaps the message, erases it, and on failure purges owned outstanding/RUDI state. A message completes successfully when it is fully sent and its `pendingPkts` reaches zero.

5.6 NACK handling

Codes	Class	Engine reaction
TRIMMED, TRIMMED_LASTHOP, NO_PKT_BUFFER, NO_GTD_DEL_AVAIL, NO_SES_MSG_AVAIL, NO_RESOURCE, PSN_OOR_WINDOW ROD_OOO	NORMAL	retransmit the NACKed PSN on the same PDC
NO_PDC_AVAIL, NO_CCC_AVAIL, NO_BITMAP	NORMAL	retransmit outstanding PSNs up to the NACKed one, lowest first spec: retry on a new PDC; modeled as retransmit after backoff
INV_DPDCID, PDC_HDR_MISMATCH, CLOSING, PDC_MODE_MISMATCH	PDC_FATAL	fail all inflight messages on the PDC explicitly, close it
INVALID_SYN	PDC_ERR	log, PDC stays active
NEW_START_PSN	NORMAL	encrypted-PDC path; payload carries the new Start_PSN (logged)

5.7 RDMA hardware bridge (`rdma-hw-uet-integration.{h,cc}`)

A standalone integration layer hooked into the SimAI `RdmaHw` receive and completion paths (`rdma-hw.cc`): per-packet UET target-state tracking (PSN recording via `UetPdsTargetState`), default-response storage with resource indices, and `CLEAR_COMMAND` generation when the outstanding-response limit is reached. It tracks UET state alongside the existing SimAI go-back-N datapath rather than replacing it; full datapath substitution is listed as future work.

5.8 Design decisions worth knowing

1. **Callback carries the destination.** `UetWireSendCb(Ptr<Packet>, uint32_t dstFa)`: without the address, ACKs cannot be routed on any topology larger than one link.
2. **Receiver-side reordering for ROD instead of NACK-and-drop.** Go-back-N collapses under loss (measured: 8 percent completion at 1 percent loss); buffering plus SACK turns each loss into exactly one retransmission (measured: 100 percent completion, goodput equal to RUD). Both behaviors are spec-permitted; the performance gap is the IRN (SIGCOMM 2018) result.
3. **Duplicates are re-ACKed, never NACKed**, and the dup check precedes the window check. An ACK loss otherwise deadlocks the sender.
4. **CACK-anchored receive window.** Anchoring at the highest PSN seen rejects retransmissions of the very packets that need recovering.
5. **Deterministic randomness everywhere** (ns-3 `RngSeedManager` + per-engine xorshift seeded by fabric address): identical seeds give byte-identical runs.
6. **Failures are loud.** Silent loss is the one unforgivable transport behavior; the benchmark's assessment prints FAIL below 100 percent completion.

6. Configuration reference (engine defaults)

Parameter	Default	Where	Meaning
MTU (<code>SetMsgMtu</code>)	4096 B	engine	fragment size for SES payload
RTO initial (<code>SetRtoInitUs</code>)	30 us (100 us in benchmark)	engine	first retransmission timeout; doubles per retry
Retry budget (<code>SetMaxRetries</code>)	7	engine	RTO retries before the message fails
MP_RANGE	1024	PSN space	acceptance window / max outstanding PSNs
Max PDCs	256	PDC manager	context pool size
Max guaranteed responses	128	PSN space	stored-response limit before CLEAR pressure
NACK retransmit cap	32	engine	per-PSN safety valve for NACK-driven retx

7. Results

Setup: 2 nodes, 200 Gbps, 100 ns delay, MTU 4096, 2000 x 8192 B messages, seed 1, loss injected on both devices, RTO 100 us, budget 7. Reproduce: `scripts/run_experiments.sh`.

Mode	Loss	Completion	Goodput	Retx	Avg latency	P99
RUD	0	100%	196.27 Gbps	0	7.4 us	14.3 us
RUD	0.1%	100%	192.96 Gbps	3	10.5 us	25.7 us
RUD	1%	100%	163.47 Gbps	63	30.5 us	151.2 us
RUD	5%	100%	100.75 Gbps	429	180.3 us	519.2 us
ROD	0	100%	196.27 Gbps	0	7.4 us	14.3 us
ROD	0.1%	100%	192.96 Gbps	3	54.7 us	187.0 us

Mode	Loss	Completion	Goodput	Retx	Avg latency	P99
ROD	1%	100%	163.47 Gbps	63	155.5 us	226.9 us
ROD	5%	100%	100.75 Gbps	429	438.4 us	796.4 us
RUDI	1% (4 KB)	100%	131.03 Gbps	35	n/a	n/a

Readings: loss costs goodput and tail latency, never correctness; RUD and ROD move the same bytes at the same rate, and the ROD latency premium is the measured cost of ordering (head-of-line blocking); delivery is byte-exact (16,384,000 of 16,384,000).

Before/after this project's audit-and-fix pass (lossless run): delivery 25.6 percent to 100 percent; end-to-end completions 0 to 100 percent; active PDCs per flow 256 to 1. Full defect list with root causes lives in `docs/REPORT.md`.

8. Troubleshooting

Symptom	Cause / fix
<code>./ns3 configure</code> fails	install compiler toolchain (<code>xcode-select --install</code> on macOS)
Binary <code>...-debug</code> not found	different build profile; reconfigure with <code>--build-profile=debug</code> or use the matching suffix
Stale-build weirdness	<code>scripts/build.sh --clean</code>
Completion below 100% at high <code>--lossRate</code>	honest behavior when the retry budget or <code>--simTime</code> is too small for the loss rate; raise either
Where is per-PDC detail?	<code>pdc_state_dump.log</code> in the invocation directory
Test suite fails after a code change	run <code>./build/scratch/ns3.36.1-uet-tests-debug</code> directly; each FAIL line names the test and assertion

9. Limitations and future work

- No congestion-control algorithm runs (`ACK_CC/ACK_CCX` state is carried, unused); the benchmark link is uncontended.
- Single path: no entropy-based packet spraying; reordering arises only from loss.
- No TSS/encryption; no switch-side packet trimming (trim NACK codes exist and are handled).
- Retransmitted packets do not set the `retx` wire flag (stored copies resent verbatim); receiver dedup makes this behaviorally invisible here.
- RUDI dedup state is unbounded (fine for simulation, not production).
- RUDI latency unmeasured (small-message SES format carries no message id).
- The `RdmaHw` bridge coexists with, rather than replaces, `SimAI`'s go-back-N datapath.

Natural next steps: credit-based CC on the existing `ACK_CC` fields; multipath spraying with reorder-tolerance measurement; full `RdmaHw` datapath substitution inside `SimAI` workloads; a systematic wire-level conformance harness.

10. Glossary

Term	Meaning
SES / PDS	Semantic / Packet Delivery Sublayer of UET
PDC	Packet Delivery Context: ephemeral connection state
IPDCID / TPDCID	PDC id assigned by initiator / target
FA / FEP	Fabric Address (IP) / Fabric End Point
PSN	Packet Sequence Number (per direction, per PDC)
Start_PSN	randomized initial PSN, $\geq 2^{16}$ from previous use
CACK_PSN	cumulative ACK: all PSNs at or below are delivered
CLEAR_PSN	highest PSN whose guaranteed-delivery state may be freed
SACK	64-bit selective-ACK bitmap based at CACK_PSN + 1
MP_RANGE	PSN acceptance window (1024)
SOM / EOM	start / end of message flags
RUD / ROD / RUDI / UUD	the four delivery modes (section 1.2)
SYN	flag set on requests until the TPDCID is learned
CP	PDS control packet (NOOP, ACK_REQUEST, CLEAR, CLOSE, PROBE, CREDIT)
RTO	retransmission timeout
Go-back-N	retransmit-everything recovery (RoCE behavior)
IRN	SIGCOMM 2018 paper showing selective retransmit beats go-back-N
SimAI	Alibaba's AI-fabric simulator; the base ns-3 tree